

CS 534: Homework #1

Submission Instructions: Similar to homework #0, the autograder supports Python 3.10. Additional packages and their versions can be found in the `requirements.txt`. Please be aware that the use of other packages and/or versions outside of those in the file may cause your homework to fail some test cases due to incompatible method calls or the inability to import the module. We have split this homework into 2 parts on Gradescope, the autograded portion and the written answer portion. *If either of the two parts is late, then your homework is late.*

1. **Enter answers for HW1-Written:** Create a single high-quality PDF with your solutions to the non-coding problems. The solutions *must be legible and each problem appropriately tagged* on Gradescope. If we must search through your entire PDF for each problem, you may lose points! Note that you must submit the code used to generate your results in the Code part of the assignment, otherwise you may not get any points for the results.
2. **Submit code to the HW1-Code Assignment:** Your submitted code must contain the following two files: `q1.py`, `elastic.py`, `README.txt`. You are welcome to submit other files but the autograder will only copy these files when running the test cases so make sure they are self-contained (i.e., capable of running standalone). Make sure you always upload *ALL* files when you (re)submit. The `README.txt` file *must contain a signed honor statement*.

Citi Bike System Data. Consider the Citi Bike trip-history dataset from New York City's Citi Bike bike-share system (Citi Bike System Data). The dataset records, for each individual bike ride, the start and end time, start and end station name/ID, start and end GPS coordinates, whether the bike was a classic or electric bike, and whether the rider was an annual member or a casual/day-pass user. We sampled a portion of the data: training data consists of 509,712 rides sampled from all of 2022 (`citibike_2022_train_processed.csv`) and test data consists of 254,059 rides sampled from all of 2023 (`citibike_2023_test_processed.csv`). We have done some simple preprocessing to remove rows with an unusable ride duration (negative, or longer than 12 hours) or missing start/end coordinates. We also calculated two basic distance measures between the start and end coordinates, `distance_km` (straight-line/haversine distance), `manhattan_km` (taxicab distance), `distance_km_sq` (haversine distance squared), and `distance_km_log1p` (log transformation of 1+ the haversine function). Your goal is to predict the duration of a ride (the `duration_min` attribute).

1. **Linear and Ridge Regression [38 pts]** A template file, `q1.py`, has been provided that stubs out the functions you will need to implement below. You are allowed to use any of the default Python libraries and available packages in `requirements.txt` for this part.
 - (a) **(Written) (3 pts)** The raw dataset includes start timestamps and station identifiers (id and name), which are not directly usable as numeric features. What do you think would be a reasonable set of numeric features to extract from these? Briefly justify your choices: what did you keep, drop, or transform, and why?
 - (b) **(Code) [5 pts]** Implement the feature engineering you described in 1a in the `preprocess_features` function. The input is a `DataFrame` from train or test. Your output should be a `DataFrame` of purely numeric features suitable for use as a design matrix. Any non-numeric or identifier columns (e.g., raw station names/IDs, timestamps) must be dropped once you've extracted whatever information you need from them. Your function should add at least several new features capturing the temporal

and/or spatial structure of a trip. This function will be used to compute your score on the class leaderboard¹ to see how good your engineered features help with prediction.

- (c) **(Code) [3 pts]** Write the function `standardize_data(trainx, testx)` which will standardize each feature by setting each feature's mean to 0 and variance to 1 (i.e., each feature will look like a Gaussian with 0 mean and unit variance) using statistics computed **from the training set only**. In other words, you should be doing: $x_{\text{standardized}} = \frac{x - \mu}{\sigma}$. Your function should return the standardized `trainx` and `testx`. Note, that `testx` may not be zero mean and unit variance as it's using the training dataset statistics.
- (d) **(Code) [3 pts]** Write `eval_linear(trainx, trainy, testx, testy)` that trains a multiple linear regression model on the training data and reports RMSE and R^2 on train and test, returned as a dictionary with keys `'train-rmse'`, `'train-r2'`, `'test-rmse'`, `'test-r2'` (e.g., `{'train-rmse': 0.5, 'train-r2': 0.2, 'test-rmse': 0.2, 'test-r2': 0.1}`).
- (e) **(Code [4 pts])** Write `eval_ridge(trainx, trainy, testx, testy, gamma)` that trains a ridge regression model ($\gamma \|\beta\|_2^2$) on the training data and reports RMSE and R^2 on train and test, returned as a dictionary with keys `'train-rmse'`, `'train-r2'`, `'test-rmse'`, `'test-r2'`.
- (f) **(Written) [9 pts]** Using `eval_ridge`, report in a table the RMSE and R^2 for train and test for 15–20 values of γ log-spaced from 10^{-4} to 10^{10} , using the standardized and unstandardized data separately. That is, run `eval_ridge` once on the output of `standardize_data`, and once on the original, unstandardized `trainx/testx`. Based on test performance, what γ would you select in each case? Does standardization change which γ looks best, or how well the model performs?
- (g) **(Written) [4 pts]** Report the test RMSE and R^2 of unregularized linear regression (1d) and compare to ridge regression at your selected γ (1f, standardized version). Does regularization meaningfully change performance on this dataset? Briefly explain your answer as to why you think that might or might not be the case.
- (h) **(Written) [7 pts]** Using the full range of γ values from 1f, generate the ridge coefficient path (coefficient value vs. γ) using the **standardized** data. Mark the optimal γ you selected (e.g., with a vertical dashed line, or a star on each coefficient's curve at that point). Based on this plot, which 2–3 features appear to matter most for predicting bike usage? Does this match your intuition about the dataset?

2. Ridge Regression Augmentation [15 pts]:

- (a) **(Written) [10 pts]** Show that the ridge regression estimates can be obtained by ordinary least squares regression on an augmented data set. We augment the centered matrix X with k additional rows with the value $\sqrt{\gamma}\mathbf{I}$ and augment $\mathbf{y}$ with k zeros. The idea is that by introducing artificial data having response value zero, the fitting procedure is forced to shrink the coefficients toward zero.
- (b) **(Written) [5 pts]** Using the dataset optimal γ you selected in 1f, construct the augmented data matrix and response vector as described above. Note that your implementation of `eval_linear` may fit the intercept by centering whatever data it's given. To keep the augmented rows' zero targets comparable regardless, center the response $\mathbf{y}$ by the training mean before augmenting (`trainy - trainy.mean()`), and `testy - trainy.mean()`). Fit ordinary least squares on this augmented set using

¹This part uses a different subsample from the CitiBike dataset.

`eval_linear`. Compare the resulting coefficients to those from `eval_ridge` fit on the same (standardized) data with the same γ . Do they match? Briefly explain why this confirms (or doesn't confirm) your derivation above.

3. Elastic Net using Stochastic Gradient Descent [47 pts]

A template file, `elastic.py`, defines a class `ElasticNet` that takes in the regularization parameters `gamma` (γ), `alpha` (α), `eta` (η) or the learning rate, the batch size (`batch` $\in [1, N]$), and `epoch` or the maximum number of epochs as parameters when creating the object (i.e., `elastic = ElasticNet(gamma, alpha, eta, batch, epoch)`). You will implement `ElasticNet` using SGD. Make sure you use the *standardized data* and not the raw data for this problem. The functions in `elastic.py` will be tested against a different training and test set (assume it has been appropriately preprocessed). For this problem, you are only allowed to use `numpy` or `scipy`. The elastic net optimization problem is:

$$\min_{\beta} f_0(\beta) = \frac{1}{2n} \|y - X\beta\|_2^2 + \gamma (\alpha \|\beta\|_2^2 + (1 - \alpha) \|\beta\|_1), \quad 0 \leq \alpha \leq 1 \quad (1)$$

where n is the number of samples in the batch or dataset the loss is being computed over.

- (a) **(Code) [5 pts]** Implement the loss objective helper function in `elastic.py`. In other words, given the coefficients, data and the regularization parameters, your function will calculate the loss $f_o(\mathbf{x})$ as shown in Eq (1).
- (b) **(Code) [8 pts]** Implement the gradient helper function in `elastic.py`. You may find it helpful to derive the update for a single training sample and to consider proximal gradient descent for the $\|\beta\|_1$ portion. As a reminder, given step size η and regularization parameter γ such that $f(\mathbf{x}) = g(\mathbf{x}) + \gamma \|\mathbf{x}\|_1$, the proximal update is:

$$\text{prox}(x_i) = \begin{cases} x_i - \gamma\eta & \text{if } x_i > \gamma\eta \\ 0 & \text{if } -\gamma\eta \leq x_i \leq \gamma\eta \\ x_i + \gamma\eta & \text{if } x_i < -\gamma\eta \end{cases}$$

- (c) **(Code) [7 pts]** Implement the Python function `train(self, x, y)` for your class that trains an elastic net regression model using stochastic gradient descent. Your function should return a dictionary where the key denotes the epoch number and the value of the loss associated with that epoch.
- (d) **(Written) [5 pts]** In the SGD update for elastic net, which part of the objective determines how large a learning rate η can be before training diverges? Is it the smooth part ($\frac{1}{2n} \|y - X\beta\|_2^2 + \gamma\alpha \|\beta\|_2^2$) or the ℓ_1 penalty term? You must briefly justify your answer using what you implemented for your proximal gradient update.
- (e) **(Written) [10 pts]** Fix $\alpha = 0.5$ and a moderate γ of your choosing (based on previous problems). Using a **fixed batch size**, train your elastic net model with each of 3 learning rates (log-spaced, e.g., $\eta \in \{0.001, 0.01, 0.1\}$) on (i) a subsample of the training data (e.g., 1-10%) and (ii) the full training set.
 - i. For each of the 6 runs, plot the training objective vs. epoch.
 - ii. Which learning rate(s) converge cleanly vs. diverge/oscillate? What learning rate will you use? Briefly justify your answer by referencing the plots.
 - iii. Does the behavior of the learning rate(s) differ between the subsample and full-sample runs?

- (f) **(Written) [12 pts]** Using the same learning rate from the previous part, train elastic net for $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$ (fixed γ , full training set).
- Report the RMSE and R^2 for each, and the number of near-zero coefficients at each α .
 - What is the optimal α ? Briefly justify your choice.
 - What does the pattern of near-zero coefficients across your three α values tell you about what each one is doing to the penalty?
 - Which of the α values should be comparable to the closed-form ridge results from 1f, and why? Also, are the results similar between SGD variant and the closed-form implementation? Why or why not?